R2.09 TP 1

Introduction à l'analyse numérique

19 mars 2026 – B. Colombel

1 De quoi s'agit-il ?

Que fait un ordinateur lorsqu'on lui demande de :
- faire une addition ?
- faire une multiplication ?
- calculer $\sqrt{2}$, $\ln 2$ ou e^2 ?
- effectuer un calcul matriciel ?
- évaluer un nombre dérivé ?
- ou plus généralement d'évaluer un résultat numérique ?

Réponse :

Il utilise un algorithme !

Une *méthode numérique* est une suite finie d'opérations arithmétiques :

$$(+, -, \times, \div)$$

permettant de résoudre un problème avec une précision arbitraire.

L'*analyse numérique* est l'étude des méthodes numériques :
- approche mathématique
- approche informatique et algorithmique
- recherche de l'algorithme le plus efficace

Les méthodes numériques s'emploient dès que des calculs sont nécessaires.
Par exemple,
- évaluation d'une fonction ou approximation d'un nombre : e, π, $\ln 2$, $\sqrt{2}$, etc.
- calcul intégral ;
- calcul matriciel ;
- résolution d'équations ;
- etc.

Dans ce TP introductif, nous allons étudier quelques problèmes d'analyses numériques :
1. évaluation d'un polynôme (algorithme de Höner) ;
2. approximation d'une fonction par un polynôme (développement de Taylor) ;
3. approximation numérique du nombre dérivé d'une fonction.

2 Algorithme de Hörner

Problématique. Considérons le polynôme $P(x) = 3x^5 - 2x^4 + 7x^3 + 2x^2 + 5x - 3$.
Si l'on veut évaluer ce polynôme en $x = a$, cela nécessite $5 + 4 + 3 + 2$ multiplications pour les puissances puis 5 multiplications avec les coefficients puis 5 additions : soit en tout 19 multiplications et 5 additions.

On peut faire des économies en terme d'opérations en suivant le schéma suivant :

$$\begin{aligned}
P(x) &= \underbrace{a_n x^n + \cdots + a_2 x^2 + a_1 x}_{\text{on met } x \text{ en facteur}} + a_0 \\
&= \left(\underbrace{a_n x^{n-1} + \cdots + a_2 x}_{\text{on met } x \text{ en facteur}} + a_1 \right) x + a_0 \\
&= \dots \\
&= ((((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3})x + \cdots + a_1)\, x + a_0
\end{aligned}$$

Dans notre exemple, nous arrivons à :

$$P(x) = ((((3x-2)x+7)x+2)x+5)x-3$$

Évaluer $P(a)$ nécessite maintenant 5 multiplications et 5 additions.

Traduction algorithmique. On a l'algorithme suivant :

Algorithme de Hörner

Entrées : un polynôme P de degré n ;
un réel α où le polynôme P sera évalué;

```
1 début
2 |  A ← a_n ;
3 |  pour j allant de n − 1 à 0 faire
4 |  |  A ← a_j + α × A ;
5 |  fin
6 fin
```

Sorties : retourner A

Dans la suite, on modélise un polynôme par la liste de ses coefficients. Ainsi, `P = [-3, 5, 2, 7, -2, 3]` correspond à la fonction polynôme de l'exemple. **Attention à l'ordre des coefficients**, `P[0]` correspond au coefficient de degré 0, `P[1]` à celui de x^1, `P[2]` à celui de x^2, etc.

Exercice 1.

1. Écrire une fonction `Poly(P, alpha)` qui prend en entrée la liste `P` des coefficients d'un polynôme et un réel `alpha` et qui retourne la valeur de $P(\alpha)$ calculée en utilsant la forme développée du polynôme.
2. Écrire une fonction `Horner(P, alpha)` qui retourne également $P(\alpha)$ mais en utilisant la méthode de Hörner.

Exemple d'utilisation.

[2]:
```
P = [-3, 5, 2, 7, -2, 3]  # Attention à l'ordre des coefficients
```

[3]:
```
%time Poly(P, 1)
```

```
CPU times: user 60 µs, sys: 7 µs, total: 67 µs
Wall time: 75.8 µs
```

[3]:
```
12
```

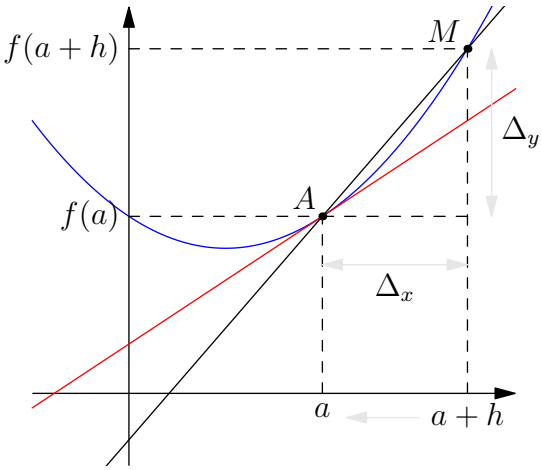
[4]:
```
%time Horner(P, 1)
```

```
CPU times: user 52 µs, sys: 0 ns, total: 52 µs
Wall time: 59.6 µs
```

[4]:
```
12
```

3 Dérivation numérique approchée

Pour savoir si une fonction f est dérivable en un point x, on considère une droite joignant les points $(x,(f(x))$ et $(x+h,f(x+h))$ de la courbe représentative de f.



Notons `deriv(f, x, h)` le coefficient directeur de cette droite. On a alors :

$$\text{deriv}(f,x,h) = \frac{\Delta_y}{\Delta_x} = \frac{f(x+h)-f(x)}{(x+h)-x} = \frac{f(x+h)-f(x)}{h}$$

Le nombre dérivé de f en x est obtenue en prenant la limite de `deriv(f, x, h)` lorsque h tend vers 0.
Informatiquement parlant, la plupart des langages n'admettent pas de système de calcul formel (contrairement à *sagemath*) et la notion de limite n'existe pas et, dans ce cas, il faut calculer le nombre `deriv(f, x, h)` avec une valeur de h *proche* de zéro afin d'obtenir une approximation du nombre dérivé.
Même si *sagemath* peut calculer des limites ou déterminer une fonction dérivée, nous allons effectuer ce travail comme si ce n'était pas le cas.

Exercice 2.

1. Écrire la fonction `deriv(f, x, h = 0.001)` et vérifier que le nombre dérivé $f'(2)$ avec $f(x) = \ln x$ est bien proche de 0,5.
2. En posant $h = 10^{-p}$ et en faisant varier p de 1 à 15 et en calculant l'erreur commise pour chaque valeur de p, déterminer expérimentalement la valeur de h qui semble donner le meilleur résultat.

Remarque. Si l'on prend h trop proche de 0, on observe une perte de précision du au fait que l'on manipule des nombres trop proches dont on perd des chiffres significatifs.

Exemples d'utilisation.

[15]:
```
deriv(ln, 2.)
```

[15]:
```
0.499875041651054
```

Exercice 3.

1. Il est anormal de privilégier un sens en faisant descendre $x+h$ vers x ; un principe général des sciences réside dans le respect de la symétrie. Coder une fonction `deriv2(f, x, h)` en remplaçant les points x et $x+h$ par $x - \frac{h}{2}$ et $x + \frac{h}{2}$.
2. Tester cette fonction sur le nombre dérivée de la fonction $x \mapsto \ln(x)$ en $x = 2$ et sa dérivée seconde (toujours en $x = 2$).

Exemples d'utilisation.

[19]:
```
h = 10**(-8)
deriv2(ln, 2., h)
```

[19]:
```
0.500001041670561
```

[20]:
```
f(x) = 1/x
deriv2(f, 2., h)
```

[20]:
```
-0.250003125051856
```

4 Développement de Taylor

4.1 Approximation par un polynôme

Considérant une fonction f définie sur un intervalle I de $\mathbb{R}$, on souhaite déterminer la meilleure approximation possible de f par un polynôme de degré n.

Polynôme de degré 1. Quel est le polynôme de degré 1 donnant la meilleure approximation de f au voisinage de x_0 ?
On cherche $P_1(x) = ax + b$ tel que :

$$\begin{cases} P_1(x_0) = f(x_0) \\ P_1'(x_0) = f'(x_0) \end{cases}$$

soit le système

$$\begin{cases} ax_0 + b = f(x_0) \\ a = f'(x_0) \end{cases} \iff \begin{cases} a = f'(x_0) \\ b = f(x_0) - x_0 f'(x_0) \end{cases}$$

Finalement, la meilleure approximation au sens donnée ci-dessus par un polynôme de degré 1 est donnée par

$$\boxed{P_1(x) = f'(x_0)(x - x_0) + f(x_0)}$$

On parle d'*approximation affine* ou de *développement de Taylor à l'ordre 1*.

Exercice 4. On prend $f(x) = \dfrac{5}{x}$ définie sur $]0; +\infty[$ et $x_0 = 2$. La fonction dérivée de f est alors $f'(x) = -\frac{5}{x^2}$.

1. Sur le même graphique représenter la fonction f et le polynôme P_1 sur l'intervalle $]\frac{1}{4}; 5]$.
2. Faire un nouveau graphique représentant l'erreur commise

$$e_1(x) = f(x) - P_1(x)$$

sur l'intervalle $]1; 3]$.

Polynôme de degré 2. Quel est le polynôme de degré 2 donnant la meilleure approximation de f au voisinage de x_0 ?
De la même façon, on cherche $P_2(x) = ax^2 + bx + c$ tel que :

$$\begin{cases} P_2(x_0) = f(x_0) \\ P_2'(x_0) = f'(x_0) \\ P_2''(x_0) = f''(x_0) \end{cases}$$

soit

$$\begin{cases} ax_0^2 + bx_0 + c = f(x_0) \\ 2ax + b = f'(x_0) \\ 2a = f''(x_0) \end{cases}$$

Le calcul formel de *sagemath* peut venir à notre aide !

[9]:
```
# d0 désigne f(x0)
# d1 désigne f'(x0)
# d2 désigne f''(x0)

a, b, c, x0, d0, d1, d2 = var('a, b, c, x0, d0, d1, d2')
solve([a*x0**2 + b*x0 +c == d0, 2*a*x0+ b==d1, 2*a==d2], a, b, c)
```

[9]:
```
[[a == 1/2*d2, b == -d2*x0 + d1, c == 1/2*d2*x0^2 - d1*x0 + d0]]
```

Ainsi,

$$\begin{aligned} ax^2 + bx + c &= \frac{1}{2}f''(x_0) \times x^2 + (-f''(x_0)x_0 + f'(x_0))x + \frac{1}{2}f''(x_0)x_0^2 - f'(x_0)x_0 + f(x_0)) \\ &= \frac{1}{2}f''(x_0)(x^2 - 2x_0x + x_0^2) + f'(x_0)(x - x_0) + f(x_0) \\ &= \frac{1}{2}f''(x_0)(x - x_0)^2 + f'(x_0)(x - x_0) + f(x_0) \end{aligned}$$

On a donc

$$\boxed{P_2(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2}$$

On parle de *développement de Taylor à l'ordre 2*.

Calcul de la dérivée seconde $f''(x_0)$. Essayons de créer une fonction qui retourne une approximation de la dérivée seconde d'une fonction f. Pour voir quelle formule utiliser, regardons le développement de Taylor à l'ordre 2 de f en $x + h$ et en $x - h$:

$$\begin{aligned} f(x + h) &\approx f(x) + hf'(x) + \frac{h^2}{2!}f''(x) \\ f(x - h) &\approx f(x) - hf'(x) + \frac{h^2}{2!}f''(x) \end{aligned}$$

Si l'on additionne ces deux lignes membre à membre, on obtient :

$$f(x + h) + f(x - h) \approx 2f(x) + h^2 f''(x)$$

En isolant $f''(x)$, on obtient l'approximation recherchée :

$$f''(x) \approx \frac{f(x + h) - 2f(x) + f(x - h)}{h^2}$$

Remarque. Cette formule est bien pratique puisqu'elle permet de calculer $f''(x_0)$ sans passer par le calcul de $f'(x_0)$.

Exercice 5. Coder la fonction `deriv_seconde(f, x, h = 0.01)` qui retourne la fonction dérivée seconde approchée de f. Tester cette fonction avec la fonction $x \mapsto \ln(x)$ pour $x = 2$.

[21]:
```
deriv_seconde(ln, 2.)
```

[21]:
```
-0.250003125052967
```

Exercice 6. Toujours en considérant la fonction $f(x) = \dfrac{5}{x}$ définie sur $]0; +\infty[$ et $x_0 = 2$.

1. Sur le même graphique représenter la fonction f, son approximation affine P_1 et son développement de Taylor à l'ordre 2 P_2 sur l'intervalle $]\frac{1}{4}; 5]$.
2. Faire apparaitre sur le même graphique les erreurs

$$e_1(x) = f(x) - P_1(x) \quad \text{et} \quad e_2(x) = f(x) - P_2(x)$$

4.2 Développement de Taylor

En poursuivant le raisonnement jusqu'à l'ordre n, on obtient :

Définition 1
Le développement de Taylor de f à l'ordre n au voisinage de x_0 est égal à :

$$f(x_0) + \frac{f'(x_0)}{1\,!}(x - x_0) + \frac{f^{(2)}(x_0)}{2\,!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n\,!}(x - x_0)^n + e_n$$

où e_n correspond à l'erreur commise et $n\,! = 1 \times 2 \times 3 \cdots \times n$.

Remarques.

1. Pour écrire cette égalité, il faut que la fonction f soit au moins dérivables n fois.
2. Cela signifie que pour x suffisamment proche de x_0, le développement de Taylor de f en x_0 est une bonne approximation de $f(x)$:

$$\boxed{f(x) \approx f(x_0) + \frac{f'(x_0)}{1\,!}(x - x_0) + \frac{f^{(2)}(x_0)}{2\,!}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n\,!}(x - x_0)^n}$$

3. On peut toujours se ramener au cas où $x_0 = 0$ et dans la plupart des cas, on utilise le développement de Taylor au voisinage de 0. Ainsi, pour x suffisamment proche de 0, on a :

$$\boxed{f(x) \approx f(0) + \frac{x}{1\,!}f'(0) + \cdots + \frac{x^n}{n\,!}f^{(n)}(0)}$$

Exemple 1. Soit $f(x) = \exp(x)$.
Comme $f'(x) = \exp(x)$ et $\exp(0) = 1$, pour x suffisamment proche de 0, on a :

$$\exp(x) \approx \exp(0) + \frac{\exp(0)}{1\,!}x + \frac{\exp(0)}{2\,!}x^2 + \cdots + \frac{\exp(0)}{n\,!} + \cdots$$

Finalement,

$$\exp(x) \approx 1 + x + \frac{x^2}{2} + \frac{x^3}{3\,!} + \cdots + \frac{x^n}{n\,!} + \cdots = \sum_{n \geqslant 0} \frac{x^n}{n\,!}$$

À noter que cette égalité est vraie pour tout $x \in \mathbb{R}$.

Exercice 7. (Approximation de $\sin\left(\frac{\pi}{3}\right)$)
Le développement de Taylor de la fonction $x \mapsto \sin(x)$ s'écrit de la manière suivante :

$$\sin(x) = x - \frac{x^3}{3\,!} + \frac{x^5}{5\,!} - \frac{x^7}{7\,!} + \cdots = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)\,!}$$

1. Sans utiliser la fonction `sin()` (bien entendu !), programmer avec une boucle `for` une fonction `sinus(x, n)` renvoyant la somme ci-dessus jusqu'à la puissance x^{2k+1} incluse.
2. Tester cette fonction avec $x = \frac{\pi}{3}$ avec $n = 10$.
   On rappelle que la valeur exacte est $\sin\left(\frac{\pi}{3}\right) = \frac{\sqrt{3}}{2}$